CS-3510 Algorithms, Spring 2026 Homework 4
Name: Pace Thomson
Collaborator(s): None

Regular total: 50 pts Bonus: 10 pts

Problem 1 (20 pts)

Suppose you are choosing between the following three algorithms:

1. (5pts) Algorithm A solves the problem by dividing it into seven subproblems of half the size, recursively solves each subproblem, and then combines the solution in linear time.
2. (5pts) Algorithm B solves the problem by dividing it into twenty-five subproblems of one fifth the size, recursively solves each subproblem, and then combines the solutions in quadratic time.
3. (10pts) Algorithm C solves problems of size n by recursively solving four subproblems of size $n - 4$, and then combines the solution in constant time.

In all cases you can assume it takes O(1) time to solve instances of size 1. What are the running times of each of these algorithms? Justify your answers. You can use the Master Theorem in your answers.

Algorithm A

Recurrence Relation:

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n)$$

Justification:

First, we must put our recurrence relation in the form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$.

- $a = 7$ (number of subproblems)
- $b = 2$ (factor by which the problem size is divided)
- $f(n) = O(n)$ (cost of dividing and combining)

Now, we calculate $n^{\log_b a}$:

$n^{\log_2 7} \approx n^{2.807}$

Next, we compare $f(n)$ to $n^{\log_b a}$. Since $f(n) = n$, we can clearly see that n is polynomially smaller than $n^{\log_2 7}$. Specifically, $f(n) = O(n^{\log_2 7 - \epsilon})$ for an $\epsilon \approx 1.807$.

This matches **Case 1** of the Master Theorem.

Running Time:

$$\Theta(n^{\log_2 7})$$

Algorithm B

Recurrence Relation:

$$T(n) = 25T\left(\frac{n}{5}\right) + O(n^2)$$

Justification:

Again, we apply the Master Theorem with the parameters from the problem:

- $a = 25$
- $b = 5$
- $f(n) = O(n^2)$

We calculate $n^{\log_b a}$:

$$n^{\log_5 25} = n^2$$

Now we compare $f(n)$ to $n^{\log_b a}$. Because $f(n) = O(n^2)$ and $n^{\log_5 25} = n^2$, we have $f(n) = \Theta(n^{\log_b a})$.

This matches **Case 2** of the Master Theorem. When the work done to combine is equal to the number of leaves in the recursion tree, we multiply by a logarithmic factor.

Running Time:

$$\Theta(n^2 \log n)$$

Algorithm C

Recurrence Relation:

$$T(n) = 4T(n-4) + O(1)$$

Justification:

The standard Master Theorem does **not** apply here because the subproblem size is reduced by a constant subtraction ($n - 4$) rather than a constant division ($\frac{n}{b}$). Instead, we can solve this by expanding the recurrence.

Let's expand the recurrence to observe the pattern:

$T(n) = 4[4T(n-8) + c] + c = 4^2T(n-8) + 4c + c$

$T(n) = 4^3T(n-12) + 4^2c + 4^1c + 4^0c$

After k steps, the equation generalizes to:

$$T(n) = 4^kT(n-4k) + c\sum_{i=0}^{k-1} 4^i$$

The base case occurs when $n - 4k \leq 1$, meaning we hit a problem of size 1 when $k \approx \frac{n}{4}$. Substituting $k = \frac{n}{4}$ into the equation:

$$T(n) = 4^{\frac{n}{4}}T(1) + c\left(\frac{4^{\frac{n}{4}} - 1}{4-1}\right)$$

Because $T(1) = O(1)$, the dominant term in this expression is $4^{\frac{n}{4}}$. We can also simplify $4^{\frac{n}{4}}$ algebraically:

$$4^{\frac{n}{4}} = (2^2)^{\frac{n}{4}} = 2^{\frac{2n}{4}} = 2^{\frac{n}{2}}$$

Running Time:

$\Theta(2^{\frac{n}{2}})$ or $\Theta(4^{\frac{n}{4}})$

Problem 2 (20 pts)

Solve the following recurrences by unrolling the recurrence. **DO NOT apply the Master Theorem**:

1. (6 pts) $T(n) = 4T(n/3) + O\left(n^{3/2}\right)$ for $n \geq 3; T(1) = 1;$
2. (6 pts) $T(n) = T(3n/4) + O(n)$ for $n \geq 4; T(1) = 1;$
3. (8 pts) $T(n) = T(n/7) + T(5n/7) + O(n)$ for $n \geq 7; T(1) = 1;$

Recurrence 1: $T(n) = 4T(n/3) + O(n^{3/2})$

Unrolling:

Let's rewrite the recurrence with a constant c:

$$T(n) = 4T\left(\frac{n}{3}\right) + cn^{3/2}$$

Now, let's unroll it by plugging $T(n/3)$ back into the equation:

$$T(n) = 4\left[4T\left(\frac{n}{9}\right) + c\left(\frac{n}{3}\right)^{3/2}\right] + cn^{3/2}$$

$$T(n) = 16T\left(\frac{n}{9}\right) + 4c\left(\frac{n}{3}\right)^{3/2} + cn^{3/2}$$

$$T(n) = 4^3T\left(\frac{n}{27}\right) + 4^2c\left(\frac{n}{9}\right)^{3/2} + 4^1c\left(\frac{n}{3}\right)^{3/2} + 4^0cn^{3/2}$$

Generalizing to k steps:

If we unroll this k times, we get:

$$T(n) = 4^kT\left(\frac{n}{3^k}\right) + \sum_{i=0}^{k-1} 4^ic\left(\frac{n}{3^i}\right)^{3/2}$$

We can factor out $cn^{3/2}$ from the summation:

$$T(n) = 4^kT\left(\frac{n}{3^k}\right) + cn^{3/2}\sum_{i=0}^{k-1}\left(\frac{4}{3^{3/2}}\right)^i$$

Finding the depth k:

The recursion hits the base case $T(1)$ when $\frac{n}{3^k} = 1$, which means $k = \log_3 n$.

The first term becomes:

$4^{\log_3 n} T(1) = n^{\log_3 4} \approx n^{1.26}$

Solving the summation:

Look at the ratio in our summation: $\frac{4}{3^{3/2}} = \frac{4}{\sqrt{27}} \approx \frac{4}{5.196}$.

Since this ratio is strictly less than 1, the geometric series converges to a constant as n goes to infinity. Therefore, the work done in the summation part is bounded by $\Theta(n^{3/2})$.

Comparing the leaf work $n^{1.26}$ to the internal work $n^{1.5}$, the internal work dominates.

Final Answer:

$T(n) = \Theta(n^{3/2})$

Recurrence 2: $T(n) = T(3n/4) + O(n)$

Unrolling:

Let's rewrite it with a constant c:

$$T(n) = T\left(\frac{3n}{4}\right) + cn$$

Unrolling it step-by-step:

$$T(n) = \left[T\left(\frac{9n}{16}\right) + c\left(\frac{3n}{4}\right)\right] + cn$$

$$T(n) = T\left(\left(\frac{3}{4}\right)^3 n\right) + c\left(\frac{3}{4}\right)^2 n + c\left(\frac{3}{4}\right) n + cn$$

Generalizing to k steps:

$$T(n) = T\left(\left(\frac{3}{4}\right)^k n\right) + \sum_{i=0}^{k-1} cn\left(\frac{3}{4}\right)^i$$

Solving the summation:

The recursion stops when $\left(\frac{3}{4}\right)^k n = 1$, making the leaf work $T(1) = O(1)$.

For the summation, we can pull out cn:

$$\text{Total Work} = O(1) + cn\sum_{i=0}^{k-1}\left(\frac{3}{4}\right)^i$$

This is a classic infinite geometric series where the ratio $r = \frac{3}{4} < 1$. The sum of an infinite decreasing geometric series is $\frac{1}{1-r}$.

$$\sum_{i=0}^{\infty} \left(\frac{3}{4}\right)^i = \frac{1}{1-\frac{3}{4}} = 4$$

So, the sum converges to exactly $4cn$.

Final Answer:

$T(n) = \Theta(n)$

Recurrence 3: $T(n) = T(n/7) + T(5n/7) + O(n)$

Unrolling:

Let the work at the root (Level 0) be cn.

At Level 1, we have two nodes. Their combined work is:

$$c\left(\frac{n}{7}\right) + c\left(\frac{5n}{7}\right) = \frac{6}{7}cn$$

At Level 2, we have four nodes. Their combined work is:

$$c\left(\frac{n}{49}\right) + c\left(\frac{5n}{49}\right) + c\left(\frac{5n}{49}\right) + c\left(\frac{25n}{49}\right) = c\left(\frac{36n}{49}\right) = \left(\frac{6}{7}\right)^2 cn$$

Generalizing to level i:

If we continue this, the total work at any full level i of the tree is:

$$\text{Work at level } i = cn\left(\frac{6}{7}\right)^i$$

Solving the summation:

The total work is the sum of all levels:

$$T(n) \leq \sum_{i=0}^{\infty} cn\left(\frac{6}{7}\right)^i$$

(Note: It's an inequality because the tree becomes unbalanced. The branches hitting $n/7$ reach the base case much faster than the $5n/7$ branches, so the lower levels are incomplete).

Because $r = \frac{6}{7} < 1$, this geometric series converges:

$$\sum_{i=0}^{\infty} \left(\frac{6}{7}\right)^i = \frac{1}{1 - \frac{6}{7}} = 7$$

So the upper bound is $7cn$, meaning $T(n) = O(n)$.

Since the work done at just the root level is already cn, the lower bound is $\Omega(n)$.

Final Answer:

$T(n) = \Theta(n)$

Problem 3 (10 pts)

You are given two sorted lists of integers of length m and n (no ties exist and sorted in increasing order). Give an $O(logm + logn)$ time algorithm for computing the k-th smallest integer in the union of the lists.
Show the pseudo-code for your proposed algorithm and include any necessary justifications. **You need to analyze** the time complexity of your algorithm to show that it is $O(logm + logn)$

1. Algorithm Justification

Instead of finding the k-th element by looking at one element at a time (which would take $O(k)$ time), we can compare the elements at the $k/2$ position in both arrays.

Let the two arrays be A and B. If we compare $A[k/2 - 1]$ and $B[k/2 - 1]$ (using 0-based indexing):

- **If $A[k/2 - 1] < B[k/2 - 1]$:** We can safely conclude that the first $k/2$ elements of A are strictly smaller than the k-th overall element. Even if all $k/2 - 1$ elements before $B[k/2 - 1]$ are smaller than $A[k/2 - 1]$, there are at most $(k/2) + (k/2 - 1) = k - 1$ elements smaller than $A[k/2 - 1]$. Thus, none of the first $k/2$ elements of A can be the k-th element. We can safely discard them and update our search to look for the $(k - k/2)$-th element in the remainder of A and the whole of B.

- **If $A[k/2-1] > B[k/2-1]$:** By the exact same logic, we can discard the first $k/2$ elements of B.

By eliminating roughly $k/2$ elements at each step, our target rank k halves with every recursive call, driving the logarithmic time complexity.

---

2. Pseudo-code

```
Algorithm FindKth(A, B, k)

#  A: Sorted list of integers
#  B: Sorted list of integers
#  k: The target rank (1-indexed)

   m = length(A)
   n = length(B)

   if m == 0:
       return B[k - 1]
   if n == 0:
       return A[k - 1]
   if k == 1:
       return min(A[0], B[0])

    i = min(m, k // 2)
    j = k - i
    if j > n:
        j = n
        i = k - j


    if A[i - 1] < B[j - 1]:
        return FindKth(A[i...m-1], B, k - i)
    else:
        return FindKth(A, B[j...n-1], k - j)
```

3. Time Complexity Analysis

At every step of the recursion, the algorithm calculates two indices and does a single constant-time $O(1)$ comparison.

- In the general case, k is reduced by $\lfloor k/2 \rfloor$ in every recursive call. The number of times you can divide k by 2 before it reaches 1 is $\log_2(k)$.
- Because the maximum possible value for k is $m+n$, the maximum depth of the recursion tree is bounded by $O(\log(m+n))$.
- Mathematically, by the properties of logarithms, $O(\log(m+n))$ is equivalent to $O(\log(\max(m,n)))$.
- Since $\log(\max(m,n)) \leq \log m + \log n$, the time complexity is strictly bounded by $O(\log m + \log n)$.

Therefore, the algorithm successfully finds the k-th element in $O(\log m + \log n)$ time.

Bonus Problem 4 (10 bonus pts)

Strassen's algorithm is a classic example of using the divide-and-conquer paradigm to reduce the asymptotic time complexity of matrix multiplication from $O(n^3)$ to approximately $O(n^{\log_2 7}) \approx O(n^{2.8074})$. In practice, Strassen's method has larger constant factors and can introduce more overhead than the standard algorithm for smaller matrices, but it remains an important illustration of how clever partitioning can speed up large matrix multiplications.

Task:

1. Implement Standard (Naïve) Matrix Multiplication
    a. Write a function **naiveMultiply(A, B)** that takes two $n \times n$ matrices A and B and returns their product C.
    b. Use the straightforward triple-nested loop approach, which runs in $O(n^3)$ time.
2. Implement Strassen's Matrix Multiplication
    a. Write a function **strassenMultiply(A, B)** that recursively applies Strassen's algorithm.
    b. Partition each $n \times n$ matrix into four $n/2 \times n/2$ submatrices.
    c. Compute the seven products according to Strassen's formula.
    d. Use these products to construct the resulting $n/2 \times n/2$ blocks of the product matrix.

e. Recursively apply Strassen's method until the subproblem size falls below a certain threshold (e.g., 2×2), at which point you may switch to the naïve multiplication for simplicity.

Compare Performance

- Generate random square matrices A and B of sizes $n = 64, 128, 256, 512$.
- For each n:
  i. Record the runtime of naiveMultiply(A, B).
  ii. Record the runtime of strassenMultiply(A, B).
  iii. Present your findings in a table with four columns:
    - Size: the value of n
    - Naïve time (s): running time of Naïve method
    - Strassen time (s): running time of Strassen method
    - Correct?: whether two output matrices are equal or not

Note: Strassen's algorithm may not show advantage in running time for matrices with size of testing. Beside showing the table. Please also **upload your source code file** to Canvas.

Output

```
~/algo  main ?1
py strassen.py
Size        | Naïve time (s)     | Strassen time (s)  | Correct?
-----------------------------------------------------------------
64          | 0.0127             | 0.0125             | Yes
128         | 0.0977             | 0.0968             | Yes
256         | 0.7719             | 0.7037             | Yes
512         | 6.8106             | 4.8826             | Yes
```

Source Code

```
import time
import random

# ==========================================
# 1. Naive Matrix Multiplication
# ==========================================
def naiveMultiply(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

# ==========================================
# Helper Functions for Matrix Operations
# ==========================================
def add_matrix(A, B):
    n = len(A)
    return [[A[i][j] + B[i][j] for j in range(n)] for i in range(n)]

def sub_matrix(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def split_matrix(matrix):
    n = len(matrix)
    mid = n // 2
    A11 = [row[:mid] for row in matrix[:mid]]
    A12 = [row[mid:] for row in matrix[:mid]]
    A21 = [row[:mid] for row in matrix[mid:]]
    A22 = [row[mid:] for row in matrix[mid:]]
    return A11, A12, A21, A22
```

```python
def combine_matrix(C11, C12, C21, C22):
    mid = len(C11)
    n = mid * 2
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(mid):
        for j in range(mid):
            C[i][j] = C11[i][j]
            C[i][j+mid] = C12[i][j]
            C[i+mid][j] = C21[i][j]
            C[i+mid][j+mid] = C22[i][j]
    return C

# ==========================================
# 2. Strassen's Matrix Multiplication
# ==========================================
def strassenMultiply(A, B, threshold=32):
    n = len(A)

    # Base Case: Switch to naive for small matrices to save overhead
    if n <= threshold:
        return naiveMultiply(A, B)

    A11, A12, A21, A22 = split_matrix(A)
    B11, B12, B21, B22 = split_matrix(B)

    M1 = strassenMultiply(add_matrix(A11, A22), add_matrix(B11, B22),
threshold)
    M2 = strassenMultiply(add_matrix(A21, A22), B11, threshold)
    M3 = strassenMultiply(A11, sub_matrix(B12, B22), threshold)
    M4 = strassenMultiply(A22, sub_matrix(B21, B11), threshold)
    M5 = strassenMultiply(add_matrix(A11, A12), B22, threshold)
    M6 = strassenMultiply(sub_matrix(A21, A11), add_matrix(B11, B12),
threshold)
    M7 = strassenMultiply(sub_matrix(A12, A22), add_matrix(B21, B22),
threshold)

    C11 = add_matrix(sub_matrix(add_matrix(M1, M4), M5), M7)
    C12 = add_matrix(M3, M5)
```

```
    C21 = add_matrix(M2, M4)
    C22 = add_matrix(add_matrix(sub_matrix(M1, M2), M3), M6)

    return combine_matrix(C11, C12, C21, C22)

# ==========================================
# 3. Performance Comparison
# ==========================================
def generate_random_matrix(n):
    return [[random.randint(1, 10) for _ in range(n)] for _ in
range(n)]

def check_correctness(C_naive, C_strassen):
    n = len(C_naive)
    for i in range(n):
        for j in range(n):
            if C_naive[i][j] != C_strassen[i][j]:
                return False
    return True

def run_benchmark():
    sizes = [64, 128, 256, 512]

    print(f"{'Size':<10} | {'Naïve time (s)':<18} | {'Strassen time
(s)':<18} | {'Correct?'}")
    print("-" * 65)

    for n in sizes:
        A = generate_random_matrix(n)
        B = generate_random_matrix(n)

        # Benchmark Naive
        start_time = time.time()
        C_naive = naiveMultiply(A, B)
        naive_time = time.time() - start_time

        # Benchmark Strassen
        start_time = time.time()
        C_strassen = strassenMultiply(A, B)
```

```
        strassen_time = time.time() - start_time

        # Check Correctness
        is_correct = "Yes" if check_correctness(C_naive, C_strassen)
else "No"

        print(f"{n:<10} | {naive_time:<18.4f} |
{strassen_time:<18.4f} | {is_correct}")

if __name__ == "__main__":
    run_benchmark()
```

AI Disclosure Statement

- Tool(s) Used:
  - Gemini
- Extent of Use:
  - Explaining concepts
  - Checking my work
  - Translating math wording so I can better understand what's going on.
  - Algorithm walkthroughs/stepping
  - Some math formatting to LaTex
- Human Contribution:
  - I went through everything the AI told me to understand it myself and reverse-engineer the process. Everything on this assignment was written by me.